

Reinforcement Learning for a Small Discrete Cilia Model

May 23, 2026

1 Current canonical Python benchmark

The project is now being developed in Python, which serves as the current canonical implementation of the small discrete 2-ball cilia-control problem. The earlier MATLAB work was extremely useful for developing intuition, testing ideas, and identifying candidate recurrent strokes, but the Python code is now the main working environment for algorithmic comparison and further research development. In particular, the Python framework makes it straightforward to compare exact dynamic programming with tabular and neural reinforcement-learning methods in a common setting.

My current Python benchmark code lives in the directory `RL_cilia_python` under the broader `RL_cilia` project folder; this directory now serves as the canonical working location for the discrete 2-ball benchmark.

As an important validation step, the current Python benchmark was checked against the previously validated MATLAB cycle and found to agree up to cyclic phase shift. This gives confidence that the Python environment preserves the essential discrete dynamics of the earlier MATLAB model while providing a cleaner basis for further computational experiments.

1.1 Discrete environment

To study the small 2-ball cilia problem in a setting where exact dynamic programming is feasible, I fixed a finite tabular environment obtained by discretizing the hinge-angle state space. The environment uses a rectangular grid in the two angular variables (ϕ_1, ϕ_2) over the allowed ranges $[-\pi/4, \pi/4]$ and $[-\pi/2, \pi/2]$, with grid sizes $[11, 21]$. For the parameters used here, this gives a total of 231 discrete states. The action space consists of all combinations of hinge increments in $\{-1, 0, 1\}$ for the two hinges, with the all-zero action removed, giving 8 discrete actions total. Thus, at each state the controller selects one of 8 possible discrete hinge updates.

The current Python environment uses the relative-angle (cumulative-angle) formulation. Conceptually, this is similar to a discrete curvature representation, since each state variable specifies a bend increment between neighboring segments and the physical segment angles are obtained by cumulative summation. By contrast, an “absolute-angle” formulation is closer to a discrete tangent-angle representation, since each state variable directly specifies the orientation of a segment in space. In this sense, the cumulative-angle formulation is more naturally aligned with bending-based and curvature-based descriptions of cilia shape, and I am using that as the current canonical representation.

The environment supports several boundary-handling rules; in the experiments reported below I used the clipping-with-penalty rule, so that actions attempting to move beyond the angular box are clipped to the nearest admissible move and incur an additional penalty. For each state-action pair, the one-step reward is the sum of a physics-based reward and, when applicable, a clipping penalty. More precisely, if a_{req} denotes the requested discrete action and a_{eff} the effective action after enforcing the angular bounds, then

$$r(s, a_{\text{req}}) = r_{\text{phys}}(s, a_{\text{eff}}) + r_{\text{clip}}(s, a_{\text{req}}),$$

where r_{phys} is the reward returned by the underlying hydrodynamic model and

$$r_{\text{clip}}(s, a_{\text{req}}) = \begin{cases} -0.1, & \text{if the requested action is clipped,} \\ 0, & \text{otherwise.} \end{cases}$$

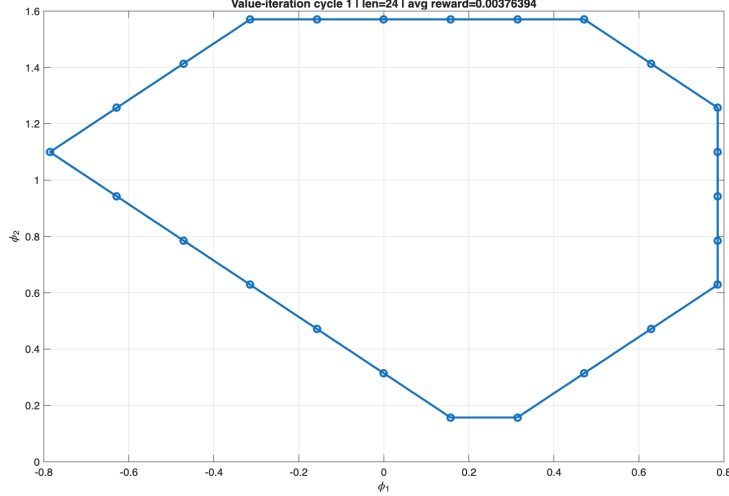


Figure 1: Phase-space plot of the length-24 cycle found by value iteration for the fixed discrete environment with $\gamma = 0.99$.

Thus, two different requested actions can produce the same next state if they clip to the same effective move, while still receiving different rewards because of the additional clipping penalty. In this way, the discrete problem is completely specified by a finite reward table $R(s, a)$ and deterministic transition table $N(s, a)$, making value iteration possible on the full state-action space. In the Python code, the rewards are also rescaled by a factor of 100, and the benchmark average rewards reported below use that rescaling.

1.2 Value iteration

For this discrete environment, I solved the discounted finite Markov decision process by value iteration with $\gamma = 0.99$. In this discrete setting, value iteration computes the optimal discounted value function $V^*(s)$ by repeatedly applying the Bellman update at every state. Concretely, for each discrete state s , the algorithm compares all admissible actions a , adds the immediate one-step reward $r(s, a)$ to the discounted value of the next state, and keeps the largest such quantity:

$$V_{k+1}(s) = \max_a (r(s, a) + \gamma V_k(N(s, a))).$$

Thus, value iteration does not search directly for a cycle. Rather, it computes, for every starting state, the best possible infinite-horizon discounted return under the model. Once this state-value function has converged, the corresponding greedy policy is obtained by choosing, at each state, the action that achieves the maximum in the Bellman update. The recurrent cycle is then a consequence of this globally optimal statewise policy. In particular, the value function depends on the starting state and on the discount factor γ , which is why two policies can produce different discounted values even if their recurrent cycles have similar geometry or even higher average reward per period.

For the current canonical benchmark, value iteration converged in 2190 iterations, reaching the stopping tolerance with

$$\|V_{k+1} - V_k\|_\infty \approx 9.985 \times 10^{-11}.$$

The resulting optimal greedy policy has a single recurrent cycle of length 24, with average reward

$$\bar{r}_{\text{VI}} \approx 0.376394.$$

The corresponding stroke in (ϕ_1, ϕ_2) -space is shown in Figure ??, and the associated per-step rewards over one period are shown in Figure ?. An important conclusion is that, for this fixed discrete environment and discount factor, the model does support a nontrivial long positive-reward periodic orbit.

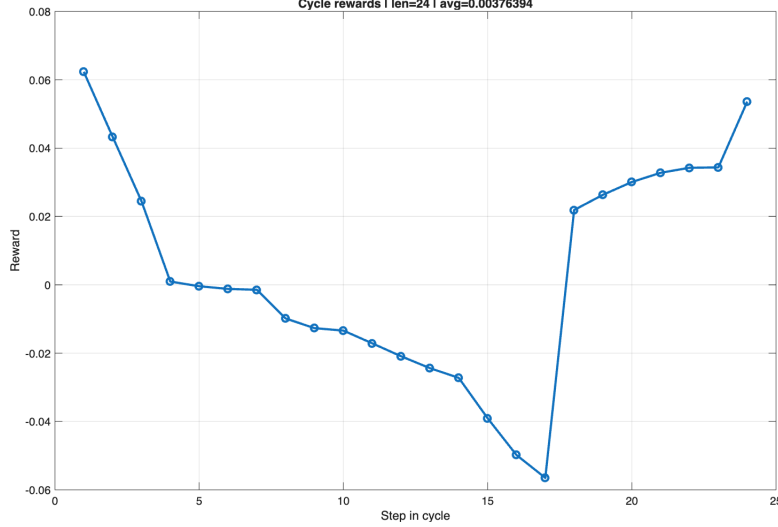


Figure 2: Per-step rewards around the value-iteration cycle. Although some portions of the cycle have negative instantaneous reward, the full cycle has positive average reward.

1.3 Comparison with tabular Q-learning, PPO, and DQN

I then compared this optimal cycle to the results of tabular Q-learning, PPO, and DQN in the same Python environment. A useful observation from tabular Q-learning is that the episode length must be large enough to expose the full long-period stroke. Earlier tabular Q-learning runs with much shorter episodes could produce shorter recurrent cycles, but once the episode length was sufficiently large (in this case 500 was sufficient), the greedy policies extracted from the learned Q -tables converged to the same length-24 cycle found by value iteration. The tabular-Q cycle agrees with the value-iteration cycle up to cyclic phase shift.

In the current canonical Python benchmark, PPO likewise recovers the same length-24 cycle as value iteration, again up to cyclic phase shift. Thus, value iteration, tabular Q-learning, and PPO all agree on the same discounted-optimal cycle for the $\gamma = 0.99$ benchmark problem.

The DQN comparison is slightly different. After correcting a reproducibility issue in the training code by explicitly seeding the environment and action-space random number generators, repeated reruns of DQN became reproducible. Under this corrected and reproducible setup, in one run DQN converges to a nearby but distinct recurrent cycle of length 25, with average reward

$$\bar{r}_{\text{DQN}} \approx 0.355267.$$

This cycle is geometrically close to the VI/tabular/PPO cycle, but it is not the same orbit.

Thus, in the current canonical Python benchmark, DQN does not recover the exact discounted-optimal cycle found by value iteration. Instead, it reproducibly converges to a nearby but suboptimal policy. This provides a useful benchmark distinction: VI, tabular Q-learning, and PPO agree on the same length-24 cycle, while DQN settles on a related but distinct length-25 alternative.

Taken together, these experiments provide a useful benchmark for later learning-based approaches. For this fixed discrete environment with $\gamma = 0.99$, value iteration identifies a unique long positive-reward cycle of length 24, and both tabular Q-learning and PPO recover that same cycle. DQN, by contrast, now appears reproducibly close but not identical.

1.4 Comparison figures and value-function diagnostics

Figure ?? compares the recurrent cycles found by value iteration, tabular Q-learning, DQN, and PPO in the discrete (ϕ_1, ϕ_2) phase plane. The tiled presentation makes the comparison easier to read than an overlaid plot. In the current canonical benchmark, the VI, tabular-Q, and PPO panels show the same length-24 cycle up to cyclic phase shift, while the DQN panel shows the nearby but distinct length-25 cycle.

Recurrent cycles in phase plane

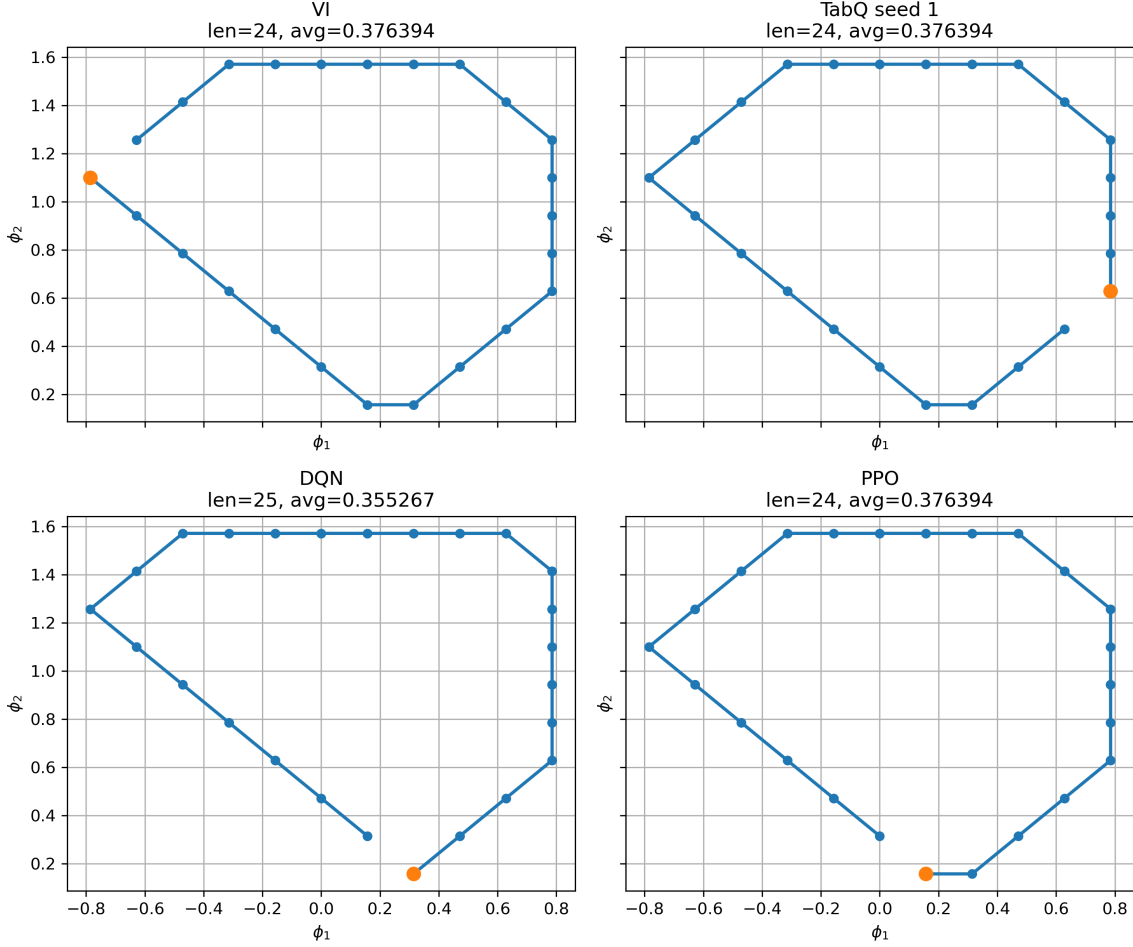


Figure 3: Tiled phase-plane comparison of the recurrent cycles found by VI, tabular Q-learning, DQN, and PPO. VI, tabular Q-learning, and PPO recover the same length-24 cycle up to cyclic phase shift, while DQN converges reproducibly to a nearby but distinct length-25 cycle.

Figure ?? shows the per-step reward traces along these corresponding cycles. Consistent with the phase-plane comparison, the reward traces for VI, tabular Q-learning, and PPO agree up to cyclic phase shift, reflecting the fact that these three methods recover the same benchmark orbit. By contrast, the DQN reward trace differs slightly, consistent with its nearby but distinct length-25 cycle and its lower average reward.

Figure ?? shows the corresponding stroke overlays in physical space. This gives a geometric view of the learned motions and confirms that the DQN orbit is still qualitatively close to the VI/tabular/PPO stroke, even though it is not identical.

Figure ?? shows a heatmap of the optimal value function V^* over the discrete (ϕ_1, ϕ_2) state grid. The value landscape is smooth and highly structured rather than noisy, with a clear gradient across the state space. In particular, some regions act as much more favorable starting states than others, in the sense that the optimal policy can reach and exploit the high-value recurrent stroke more quickly from those states. This provides a useful geometric picture of the discounted control problem: the optimal policy is not merely selecting a good local action at each state, but is organizing the entire state space into a coherent flow toward the optimal long-period orbit.

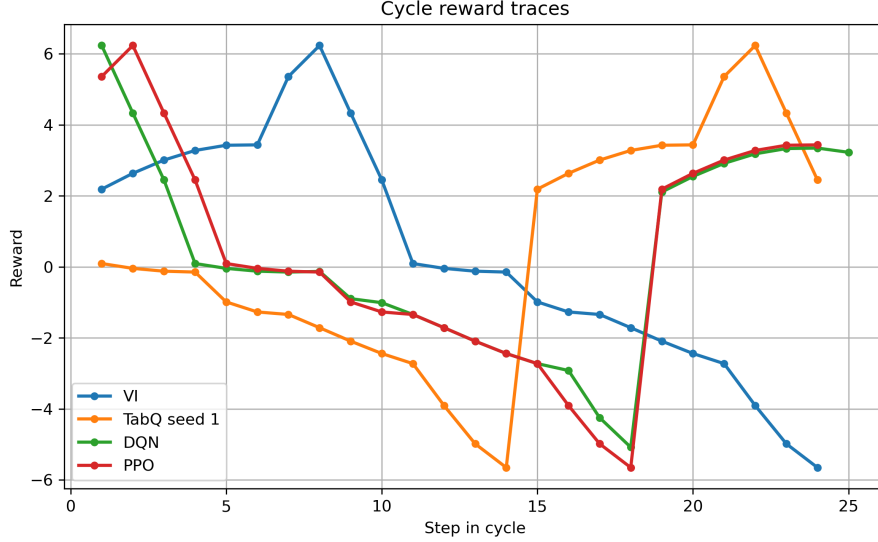


Figure 4: Per-step reward traces along the recurrent cycles found by VI, tabular Q-learning, DQN, and PPO. The VI, tabular-Q, and PPO traces agree up to cyclic phase shift, while the DQN trace differs slightly, consistent with its distinct length-25 cycle.

2 Discount-factor sweep for value iteration

To explore how the discounted objective influences the optimal recurrent stroke, I ran value iteration for a range of discount factors

$$\gamma \in \{0.900, 0.950, 0.980, 0.990, 0.992, 0.995, 0.997, 0.999\},$$

using the same discrete environment and the same convergence tolerance in each case. For each value of γ , I extracted the best recurrent cycle under the resulting optimal policy and recorded its length and average reward.

The resulting picture is quite structured. As γ increases, the optimal cycle becomes longer and its average reward per cycle increases. For smaller γ , the optimal policy favors short, relatively local recurrent motions. As γ approaches 1, the optimal policy increasingly favors larger and more elaborate strokes. This is consistent with the interpretation of γ as controlling how much the policy values future reward: larger γ allows the optimizer to accept a longer transient or a more extended cycle in exchange for improved long-term performance.

Figure ?? shows the optimal cycles in the phase plane for all values of γ . The progression is not random: the cycles appear to evolve through a coherent family of shapes as γ increases. In particular, the higher- γ cycles ($\gamma = 0.99$ and above) are closely related geometrically, although not identical. Figure ?? shows corresponding stroke plots in physical space for a selected subset of γ -values. These plots confirm that the changes seen in the phase plane correspond to meaningful changes in the underlying cilia motion.

The summary plots in Figures ?? and ?? make the main trends especially clear. The cycle length increases from 4 at $\gamma = 0.9$ to 24 at $\gamma = 0.99$, and then to 25 or 26 for the largest values of γ considered. The average reward also increases steadily across this sweep. The slight non-monotonicity in cycle length at the largest values (for example, 26 at $\gamma = 0.997$ and 25 at $\gamma = 0.999$) suggests that the family of optimal cycles may undergo small structural changes in this near-undiscounted regime, even while the overall performance trend remains monotone.

The reward traces in Figure ?? are also informative. For larger γ , the optimal cycles tend to include larger positive rewards early in the cycle together with more substantial negative portions later on. This suggests that the higher- γ policies are willing to tolerate more locally unfavorable moves as part of a longer cycle that achieves better long-run performance. In other words, increasing γ does not merely stretch the

Stroke overlays in physical space

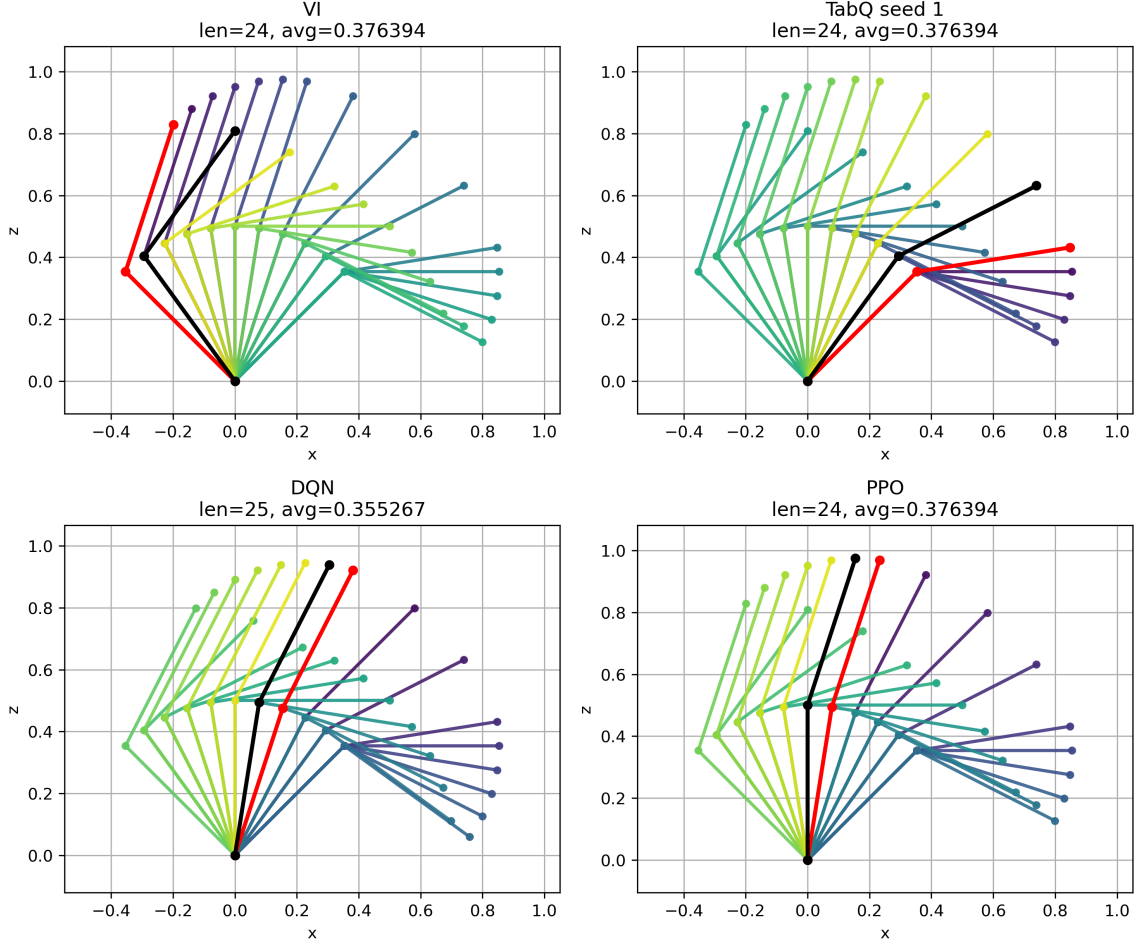


Figure 5: Tiled stroke overlays in physical space for the cycles found by VI, tabular Q-learning, DQN, and PPO. The DQN stroke remains geometrically close to the VI/tabular/PPO stroke, but it is not identical.

same solution; it changes the balance between short-term and long-term reward in a way that produces qualitatively different optimal recurrent motions.

Overall, this sweep suggests that the choice of discount factor is not a minor numerical detail in this problem. Instead, it acts as a meaningful modeling parameter that shapes the nature of the optimal stroke. This makes γ -dependence an important part of understanding the control landscape for this cilia locomotion problem.

3 Phase II direction: beyond the exact benchmark

The Phase I discrete benchmark provides a useful exact reference problem, but it is not the final scientific target of the project. Its main role is to validate the environment, compare methods in a controlled setting, and identify families of good recurrent strokes. The next phase should therefore shift emphasis away from exact agreement with the discounted value-iteration solution and toward understanding how reinforcement-learning methods behave when the exact benchmark is no longer the main objective.

A natural first step is to continue using the small two-ball cilia problem as a controlled method-development testbed, but now with a different emphasis. In particular, Phase II will include systematic

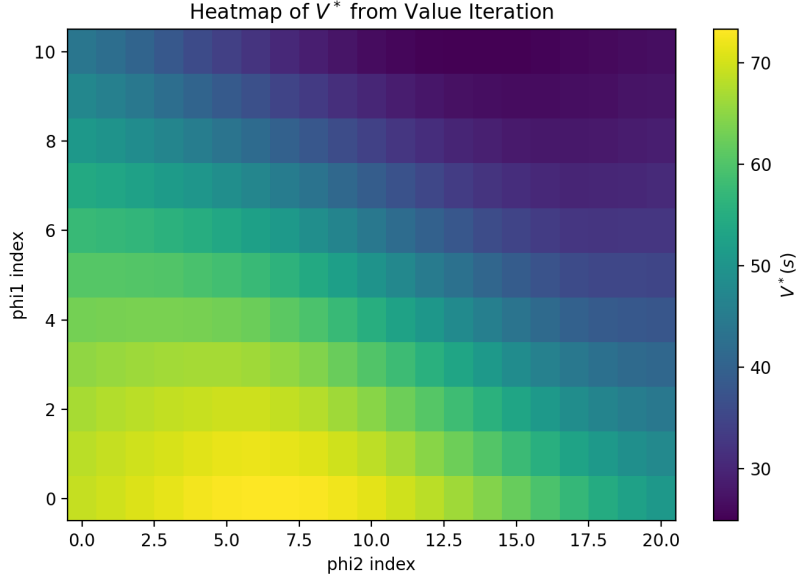


Figure 6: Heatmap of the optimal value function V^* on the discrete (ϕ_1, ϕ_2) grid. The smooth gradient structure shows how the optimal policy organizes the state space toward the high-value recurrent stroke.

exploration of DQN and PPO on this problem, with attention to the recurrent cycles they discover, the effect of training length and hyperparameters, and the distinction between discounted-optimal cycles and cycles with larger average reward per stroke. This keeps the problem simple enough to interpret while allowing the project to begin focusing on the more relevant long-term question: how to use RL to discover good beating strokes in settings where value iteration and tabular methods are no longer available.

From there, the next expansion would be to move toward larger and more flexible models, such as three- or four-ball cilia, or eventually continuous/low-mode descriptions of cilia shape. In that regime, exact dynamic programming will disappear, and methods such as PPO and DQN will need to be judged directly by the quality and robustness of the recurrent strokes they find.

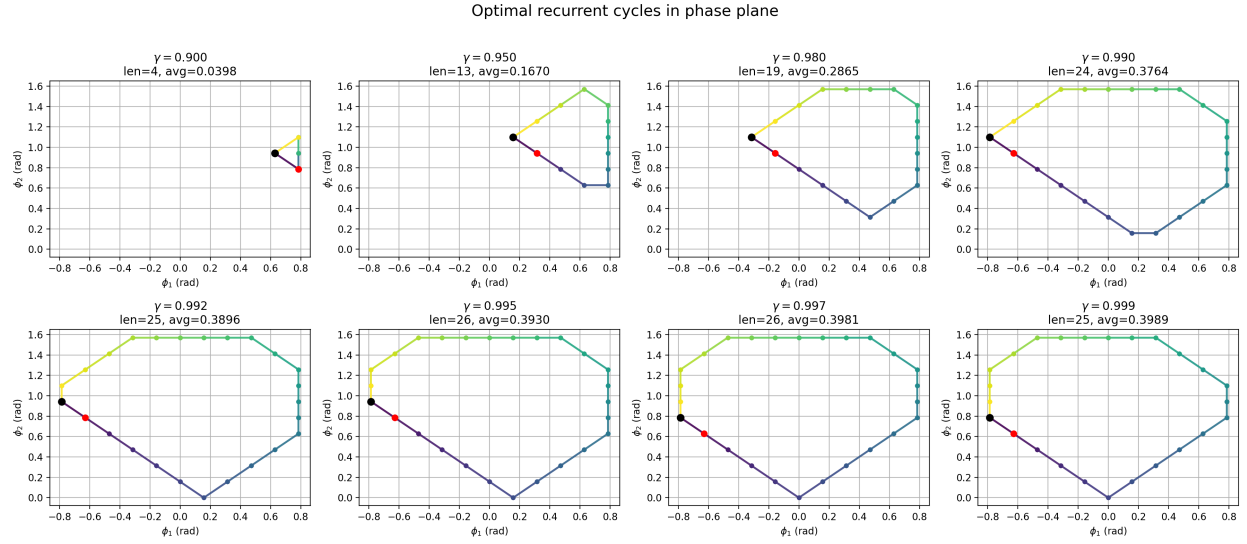


Figure 7: Optimal recurrent cycles in the phase plane for the value-iteration policy at each discount factor γ . As γ increases, the optimal cycle becomes longer and moves through a coherent family of larger recurrent orbits.

Stroke plots for selected discount factors

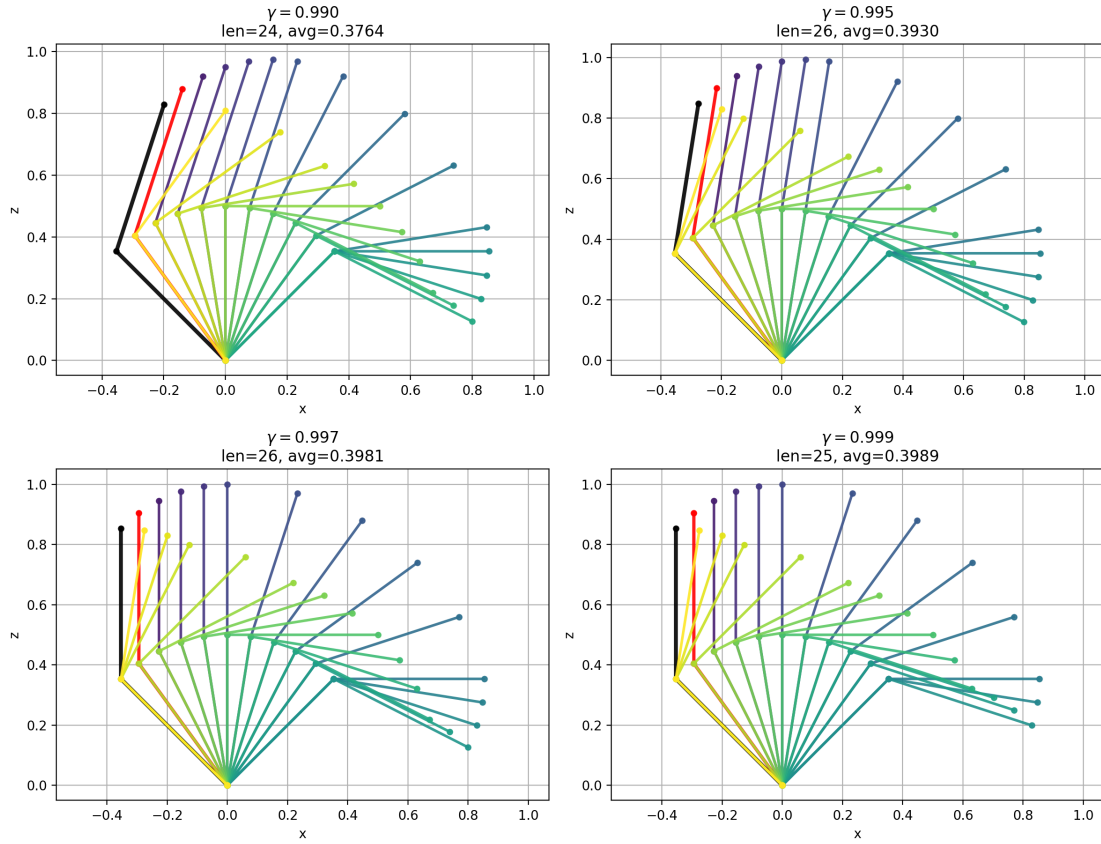


Figure 8: Stroke plots in physical space for selected discount factors. These plots show how the phase-plane changes translate into changes in the geometry of the underlying cilia motion.

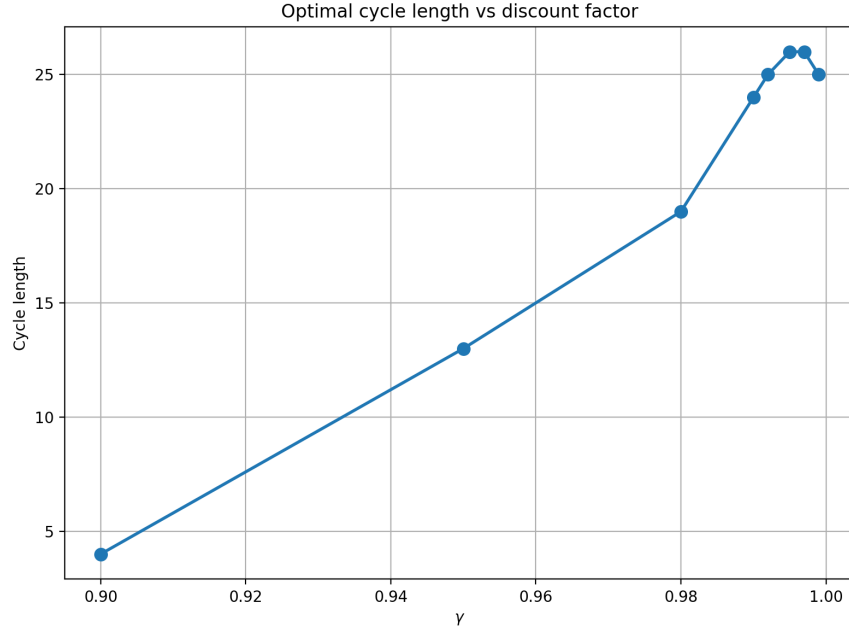


Figure 9: Length of the optimal recurrent cycle as a function of the discount factor γ . The optimal cycle length grows substantially as γ increases.

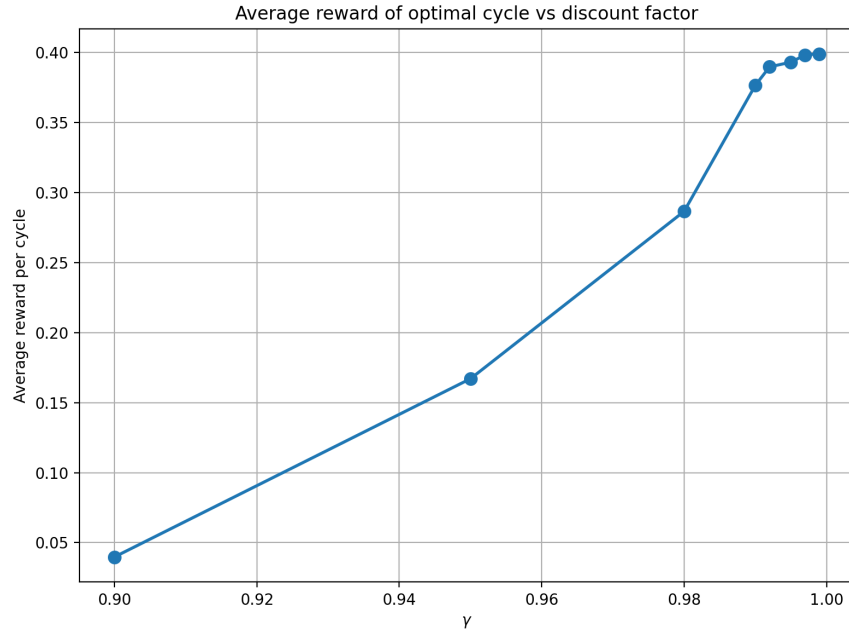


Figure 10: Average reward per cycle as a function of the discount factor γ . Larger γ leads to higher-performing recurrent cycles in this environment.

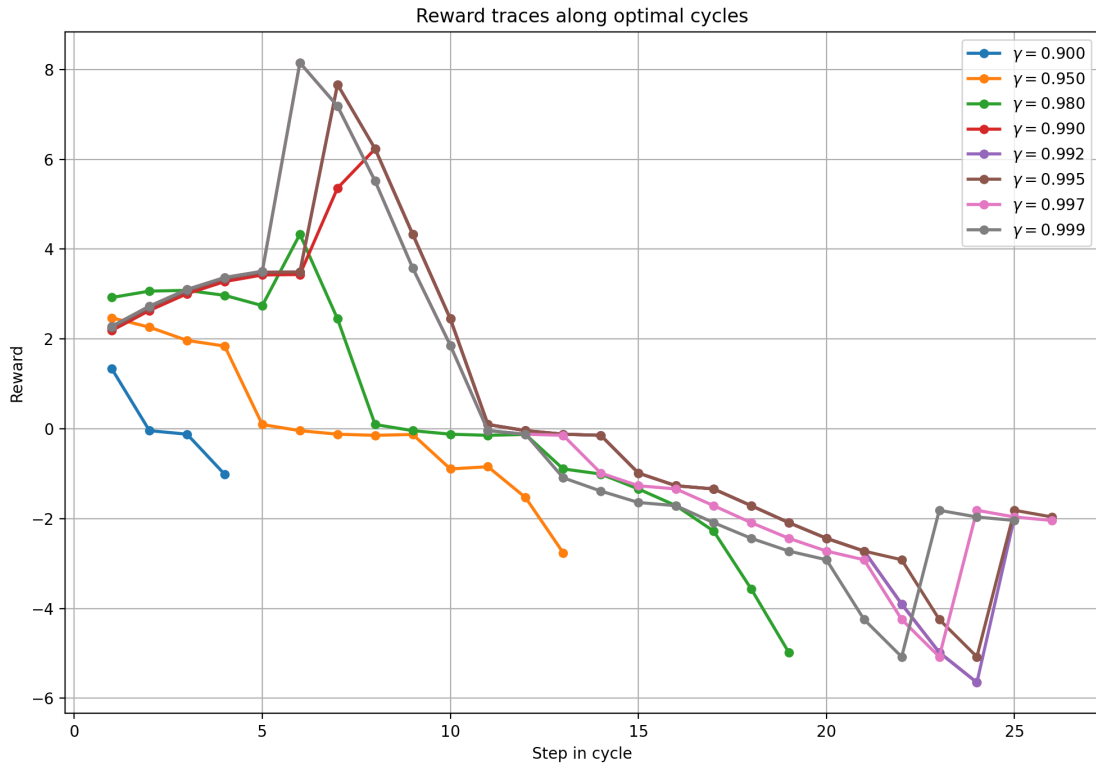


Figure 11: Reward traces along the optimal recurrent cycles for each discount factor γ . Higher- γ cycles exhibit a different balance of positive and negative rewards, reflecting the greater importance of long-horizon performance in the discounted objective.